

Meenakshi Manoj Nair

Data Science

**Fashion Image Classification & Personality Mapping
& Personality Mapping**

Introduction

Fashion is a powerful form of self-expression, often reflecting aspects of an individual's personality. In this project, we address the problem of image classification in the fashion domain by building a deep learning model to categorize clothing images into distinct fashion styles.

The primary objective is not only to classify fashion images but also to explore the possibility of mapping these categories to broader "fashion personalities" such as Minimalist, Casual, Elegant, and Bold. This adds an interpretative layer to traditional image classification, making the results more meaningful.

This work has practical applications in multiple areas. In e-commerce, it can enable personalized recommendations based on user style preferences. In fashion analytics, it can assist in identifying trends and consumer behavior. Additionally, it can support AI-based styling systems that align outfit suggestions with individual personality traits.

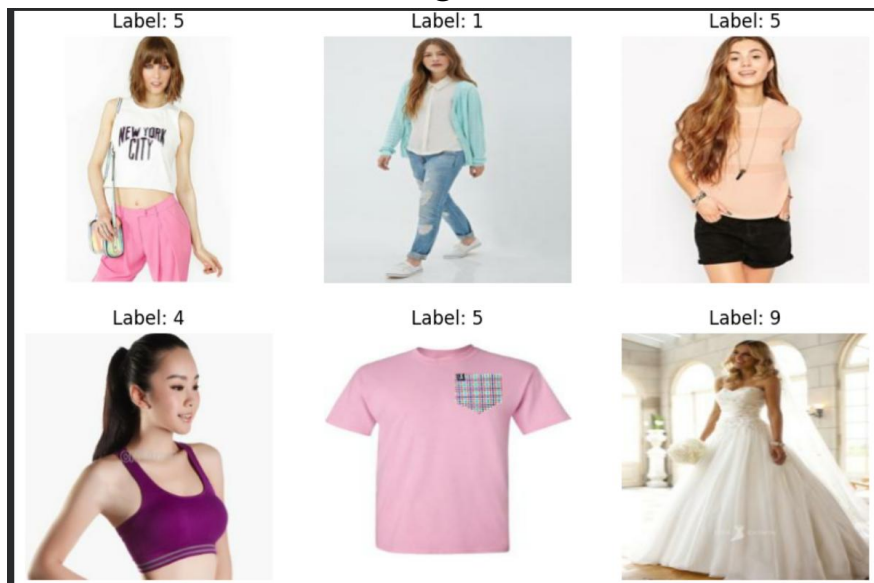
By combining computer vision techniques with interpretability through personality mapping, this project demonstrates how deep learning can bridge the gap between visual data and human expression.

Dataset Description

For this project, a subset of the DeepFashion dataset was used, which is a large-scale benchmark for fashion image understanding. The original dataset contains approximately 800,000 images with diverse clothing categories and attributes, making it well-suited for training deep learning models.

Due to computational constraints and the need for efficient experimentation, a smaller subset was curated. The final dataset consists of 10,000 images selected from the top 10 selected fashion categories.

To ensure fair representation across classes, the dataset was balanced by sampling 1,000 images per category. This balanced distribution helps prevent model bias toward dominant classes and enables more reliable evaluation across all categories.



Data Preprocessing

Before training the model, several preprocessing steps were applied to ensure data quality and consistency:

Removing Invalid Images:

Invalid or missing image paths were identified and removed to ensure that only valid data was used during training.

Dataset Balancing:

A balanced dataset was created by sampling 1,000 images per class across 10 categories, resulting in 10,000 images. This prevents bias toward any particular class and ensures fair model learning.

Train/Test Split:

The dataset was split into 80% training and 20% testing sets. Stratified sampling was used to maintain equal class distribution in both sets.

Image Resizing:

All images were resized to 224×224 pixels to maintain a consistent input size for the CNN model.

Normalization:

Pixel values were scaled from the range $[0, 255]$ to $[0, 1]$ by dividing by 255. This improves training stability and helps the model converge faster.

These preprocessing steps ensure that the dataset is clean, balanced, and suitable for efficient deep learning model training.

CNN Model Architecture

For this image classification task, a custom Convolutional Neural Network (CNN) was designed due to its strong ability to learn spatial features directly from image data. CNNs are particularly effective in capturing hierarchical patterns, starting from simple features such as edges and textures in early layers to more complex structures in deeper layers. This makes them well-suited for fashion image classification, where subtle visual differences play an important role.

The model takes input images of size $224 \times 224 \times 3$ and processes them through a series of convolutional and pooling layers. The first convolutional layer uses 32 filters, followed by a second and third layer with 64 and 128 filters respectively, all using 3×3 kernels and ReLU activation. Each convolutional layer is followed by a MaxPooling layer, which reduces the spatial dimensions of the feature maps while retaining the most important information. This progressive feature extraction allows the model to learn increasingly abstract representations of the input images.

After the convolutional blocks, a Global Average Pooling layer is applied to reduce the feature maps into a compact representation. This approach helps in reducing the number of parameters and minimizes overfitting compared to traditional flattening methods. The extracted features are then passed through a fully connected dense layer with 128 units and ReLU activation, followed by a Dropout layer with a rate of 0.3 to further prevent overfitting.

Finally, the output layer consists of 10 neurons with a softmax activation function, corresponding to the 10 fashion categories. This layer produces a probability distribution over all classes, allowing the model to predict the most likely category for each input image.

Overall, the architecture follows a structured pipeline of feature extraction, dimensionality reduction, and classification, enabling the model to effectively learn and distinguish between different fashion styles.

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),

    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.GlobalAveragePooling2D(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.3),

    layers.Dense(10, activation='softmax')
])
```

Training Strategy

To train the CNN model effectively, a structured training strategy was adopted by carefully selecting the optimizer, loss function, and training parameters. The Adam optimizer was used due to its efficiency and ability to adapt learning rates during training, which helps the model converge faster and more reliably.

Since this is a multi-class classification problem with integer-encoded labels, sparse categorical crossentropy was chosen as the loss function. This is well-suited for problems where each input belongs to exactly one class and the model outputs a probability distribution across multiple categories.

The model was trained in batches of 16 images, which provided a balance between computational efficiency and stable learning. A smaller batch size also introduces slight variability in gradient updates, which can improve generalization. Training was performed for a maximum of 50 epochs, allowing sufficient time for the model to learn patterns from the data.

To prevent overfitting and avoid unnecessary training, an early stopping mechanism was applied. This monitored the validation loss and stopped training if no improvement was observed for three consecutive epochs. The best-performing model weights were restored automatically, ensuring optimal performance on unseen data.

```

Epoch 1/50
125/125 ————— 33s 155ms/step - accuracy: 0.1124 - loss: 2.3000 - val_accuracy: 0.1330 - val_loss: 2.2894
Epoch 2/50
125/125 ————— 17s 128ms/step - accuracy: 0.1341 - loss: 2.2786 - val_accuracy: 0.1360 - val_loss: 2.2765
Epoch 3/50
125/125 ————— 16s 122ms/step - accuracy: 0.1475 - loss: 2.2614 - val_accuracy: 0.1415 - val_loss: 2.2626
Epoch 4/50
125/125 ————— 17s 125ms/step - accuracy: 0.1579 - loss: 2.2486 - val_accuracy: 0.1580 - val_loss: 2.2685
Epoch 5/50
125/125 ————— 17s 122ms/step - accuracy: 0.1660 - loss: 2.2362 - val_accuracy: 0.1705 - val_loss: 2.2403
Epoch 6/50
125/125 ————— 17s 126ms/step - accuracy: 0.1795 - loss: 2.2195 - val_accuracy: 0.1750 - val_loss: 2.2381
Epoch 7/50
125/125 ————— 17s 126ms/step - accuracy: 0.1879 - loss: 2.1998 - val_accuracy: 0.2070 - val_loss: 2.2062
Epoch 8/50
125/125 ————— 16s 119ms/step - accuracy: 0.2050 - loss: 2.1764 - val_accuracy: 0.1775 - val_loss: 2.2215
Epoch 9/50
125/125 ————— 22s 128ms/step - accuracy: 0.2097 - loss: 2.1686 - val_accuracy: 0.2220 - val_loss: 2.1651
Epoch 10/50
125/125 ————— 17s 131ms/step - accuracy: 0.2210 - loss: 2.1338 - val_accuracy: 0.2225 - val_loss: 2.1511
Epoch 11/50
125/125 ————— 20s 124ms/step - accuracy: 0.2295 - loss: 2.1155 - val_accuracy: 0.2100 - val_loss: 2.1424
Epoch 12/50
125/125 ————— 17s 126ms/step - accuracy: 0.2365 - loss: 2.0962 - val_accuracy: 0.2325 - val_loss: 2.1059
Epoch 13/50
125/125 ————— 17s 127ms/step - accuracy: 0.2460 - loss: 2.0795 - val_accuracy: 0.2335 - val_loss: 2.1088
Epoch 14/50
125/125 ————— 19s 120ms/step - accuracy: 0.2486 - loss: 2.0519 - val_accuracy: 0.2430 - val_loss: 2.0885
Epoch 15/50
125/125 ————— 16s 120ms/step - accuracy: 0.2590 - loss: 2.0423 - val_accuracy: 0.2360 - val_loss: 2.1034
Epoch 16/50
125/125 ————— 16s 120ms/step - accuracy: 0.2651 - loss: 2.0292 - val_accuracy: 0.2615 - val_loss: 2.0782

```

Experiments & Challenges

During model development, several approaches were tested to improve performance. Initially, a pre-trained EfficientNet model was used, but it resulted in low accuracy ($\sim 10\%$), close to random chance. Due to this poor performance, a custom CNN was adopted, which provided better control and improved results.

Data augmentation techniques were also experimented with to enhance generalization. However, they significantly increased training time without noticeable performance gains, so they were excluded from the final model.

Training time itself was a challenge due to the relatively large image size (224×224), which increased computational cost. Additionally,

class imbalance in the original dataset was addressed by creating a balanced subset with equal samples per class.

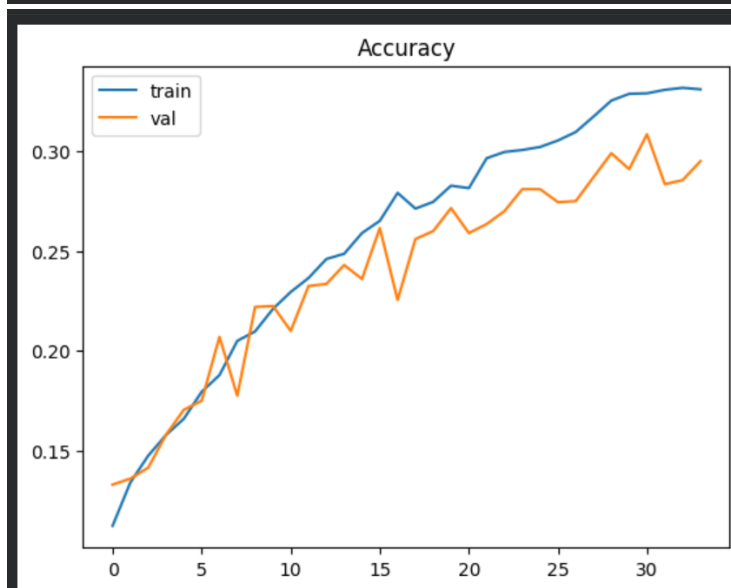
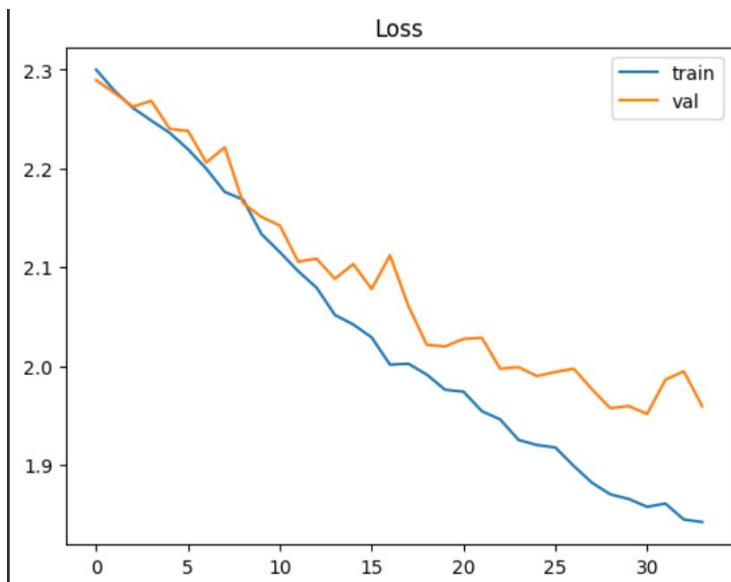
Overall, these experiments helped guide the final model design by balancing performance, efficiency, and practicality.

Results

After training the CNN model, a final test accuracy of approximately 31% was achieved. This performance is higher than random chance (10% for 10 classes), indicating that the model has learned meaningful patterns from the data.

However, moderate accuracy also reflects the challenges of fashion image classification. Many categories share similar visual features, making it difficult for the model to clearly distinguish between them. Additionally, variations in image conditions such as pose, lighting, and background further impact performance.

Overall, this result serves as a baseline, demonstrating that while the model is capable of learning, there is significant scope for improvement through more advanced techniques such as transfer learning and improved model design.



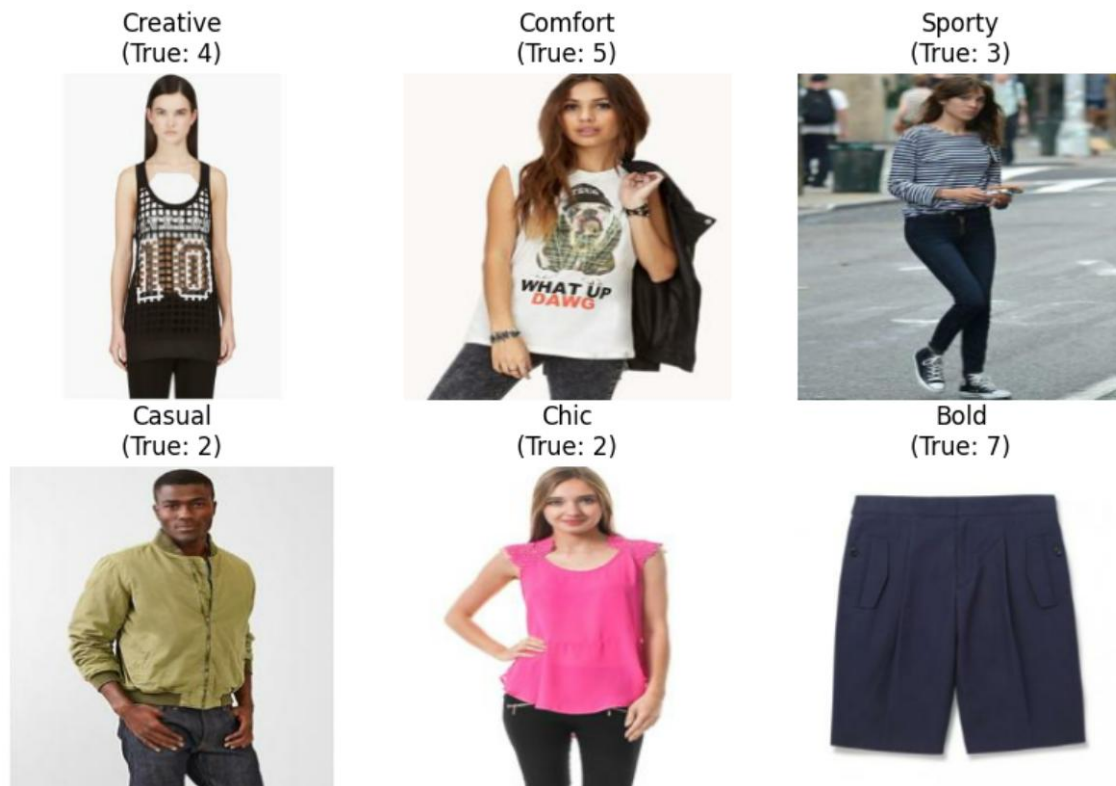
Personality Mapping

Beyond classification, this project also explores mapping predicted fashion categories to broader “fashion personality” labels. A personality map was created to translate model outputs into more interpretable terms such as Minimalist, Casual, Elegant, and Bold.

This mapping helps bridge the gap between raw model predictions and meaningful real-world interpretation. Instead of viewing outputs as just numerical class labels, they can be understood as style preferences or personality traits, making the results more intuitive.

For example, in an application setting, users frequently interacting with items classified as Elegant or Chic could be associated with a particular fashion personality, enabling more personalized recommendations or styling suggestions.

It is important to note that this mapping is interpretive rather than learned by the model. The assignment of personality labels to fashion categories is based on domain intuition and design choices, and may vary depending on context. Despite this subjectivity, it adds a useful semantic layer that enhances the practical value of the model's predictions.



Evaluation

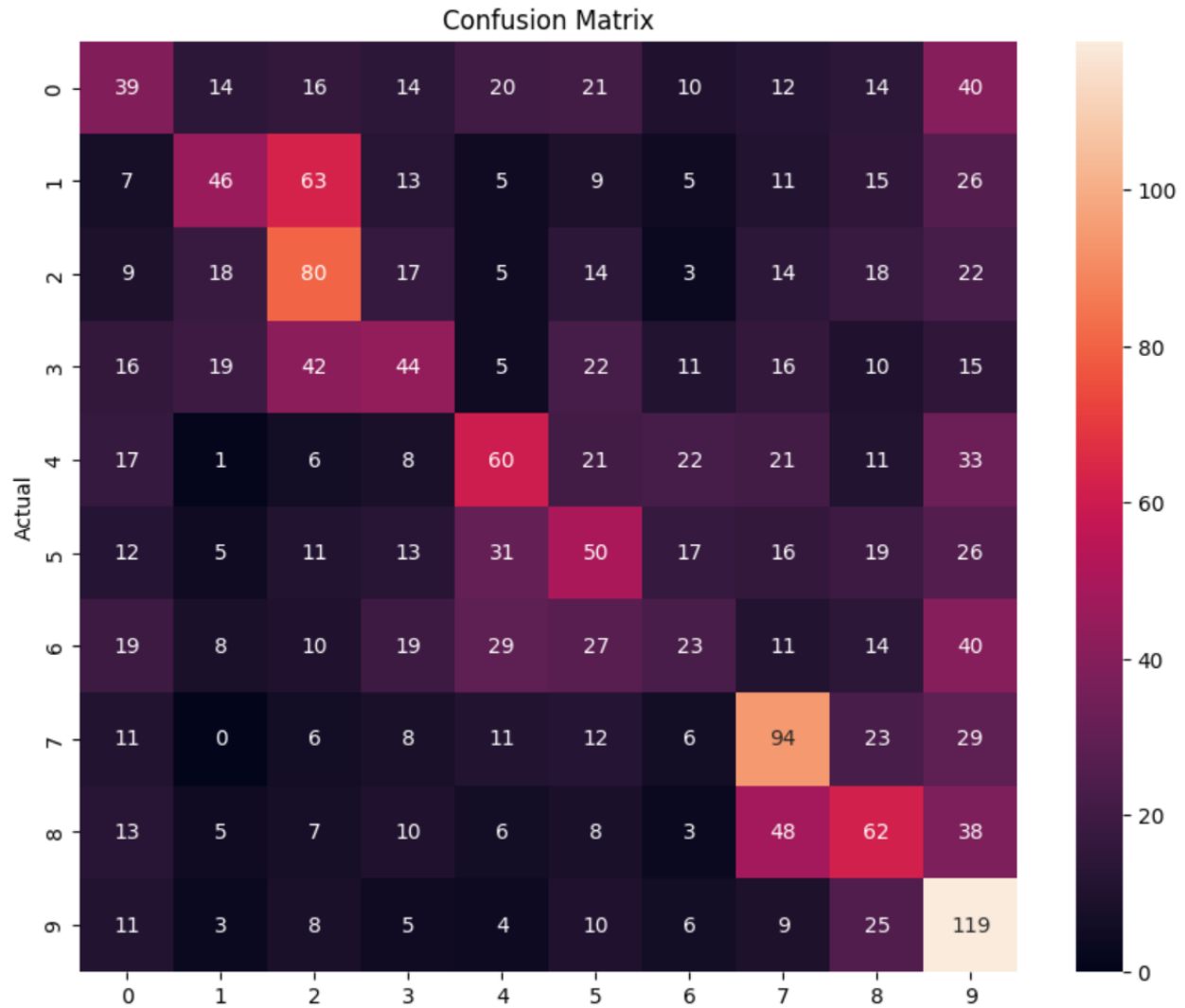
The model was evaluated using a classification report and confusion matrix, with an overall test accuracy of 31%.

The classification report shows that performance varies across categories. Classes such as Bold, Creative, and Elegant performed relatively better, with higher recall and F1-scores. For example, Bold achieved an F1-score of 0.42, indicating the model could identify this class more effectively compared to others. However, even in these cases, precision and recall were not consistently high, showing room for improvement.

In contrast, classes like Simple and Minimalist showed weaker performance, with low F1-scores (around 0.15–0.22). This suggests that the model struggles to distinguish these categories due to their visual similarity with other classes.

The confusion matrix further supports these findings by showing frequent misclassifications between similar categories such as Simple, Minimalist, and Comfort. On the other hand, more visually distinctive classes like Bold and Creative were comparatively easier for the model to classify correctly.

Overall, the variation in performance highlights the challenges of fine-grained fashion classification and indicates that a basic CNN has limitations in capturing subtle inter-class differences.



UMAP Visualization

UMAP (Uniform Manifold Approximation and Projection) is a dimensionality reduction technique used to project high-dimensional feature vectors into a 2D space for visualization while preserving their structure.

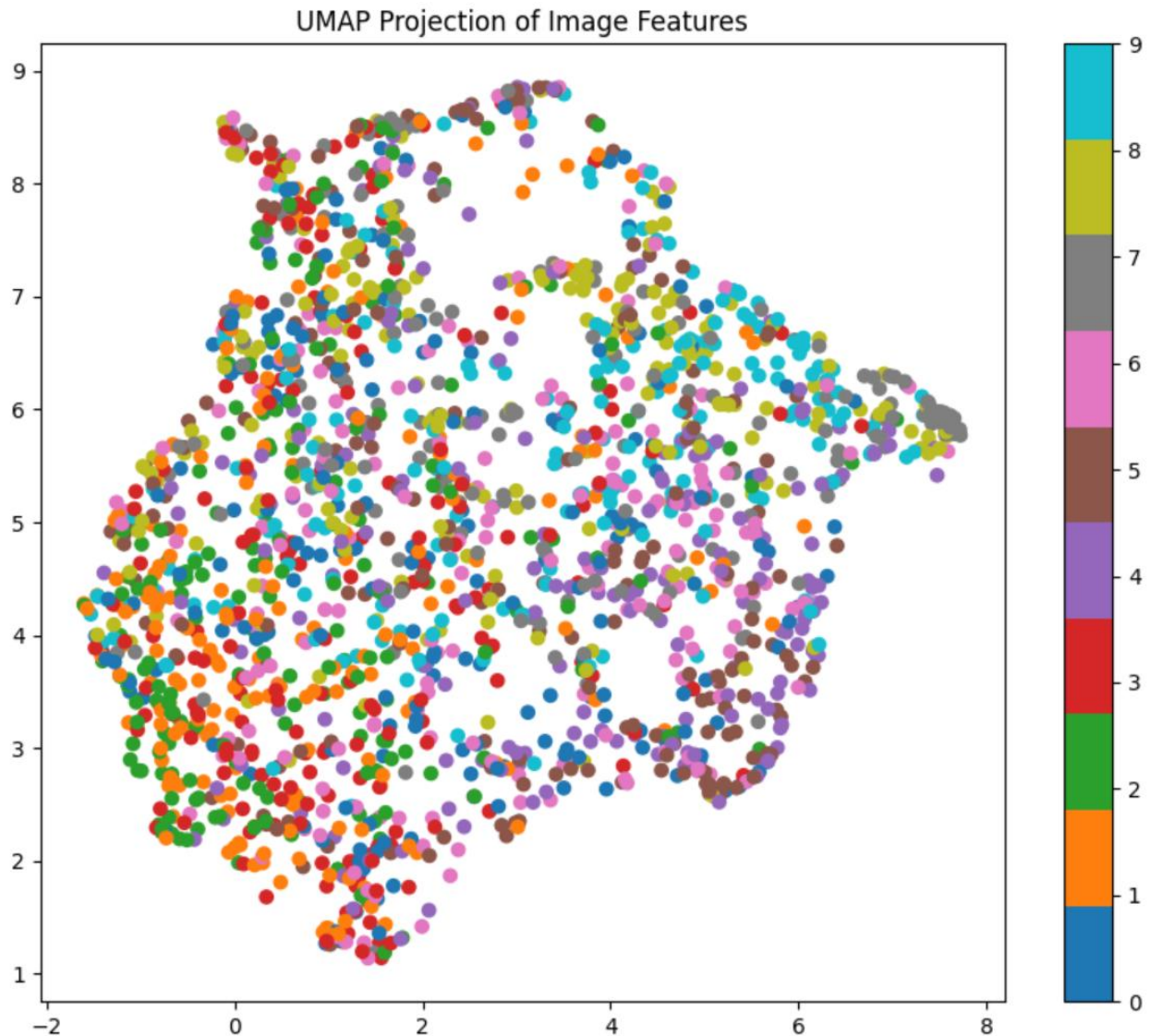
In this project, UMAP was applied to feature representations extracted from the CNN's intermediate layers (after global average pooling).

These feature vectors capture high-level visual patterns learned by the model and are typically high-dimensional, making direct visualization difficult.

The main purpose of using UMAP was to analyze how well the model separates different fashion categories in the learned feature space. Ideally, each class should form a distinct cluster, indicating strong feature discrimination.

The visualization shows partial clustering of some categories, suggesting that the model has learned meaningful representations for certain classes. However, there is also significant overlap between several categories, especially those with similar visual characteristics such as Simple, Minimalist, and Sporty. This overlap aligns with the moderate classification performance observed earlier.

Overall, the UMAP plot provides a qualitative understanding of the model's internal feature space and confirms that while the model captures some structure, it still struggles to fully separate visually similar fashion categories.



Key Insights

This project demonstrated an end-to-end deep learning pipeline for fashion image classification, from data preprocessing to model evaluation. The custom CNN was able to learn meaningful visual features and perform better than random chance. Techniques such as early stopping helped improve generalization, while UMAP and confusion matrix visualizations provided useful insights into model

behavior. The personality mapping added an interpretable layer to the predictions.

However, the model achieved a moderate accuracy of around 31%, showing limitations in distinguishing fine-grained fashion categories. Significant confusion was observed between visually similar classes such as Simple, Minimalist, and Sporty, and UMAP also showed overlapping feature clusters.

Overall, the project highlights the challenges of fine-grained image classification and the importance of more advanced models, better feature learning, and visualization tools for understanding performance.

Limitations & Future Work

This project demonstrates a baseline approach to fashion image classification using a custom CNN, but several limitations were observed.

The model achieved a moderate accuracy of approximately 31%, indicating limited ability to capture fine-grained differences between fashion categories. Significant confusion was observed between visually similar classes such as *Simple*, *Minimalist*, and *Sporty*, which contributed to overall performance limitations. Additionally, the relatively simple CNN architecture restricts its ability to learn highly discriminative features from complex fashion images.

To improve performance, several future directions can be explored. Transfer learning using pre-trained models such as MobileNet, ResNet, or VGG can significantly enhance feature extraction and improve accuracy. More advanced data augmentation techniques can also

improve model generalization by increasing data diversity. Further improvements can be achieved through hyperparameter tuning and the use of ensemble methods. Finally, incorporating attention mechanisms may help the model focus on more relevant regions of the image, improving fine-grained classification.

Overall, these improvements can help build a more robust and accurate system for fashion classification and personality-based recommendation.